

Gün 13 — Pipeline'ın FastAPI'ye Asenkron Entegrasyonu

Hedef

1-2. haftada yazılan `scripts/` altındaki analiz/extraction modüllerini (`detect_trailer` , `size_analysis` , `entropy` , `extract` , `video_metadata`) `POST /api/v1/analyze` endpoint'ine bağlamak; CPU-yoğun işlemleri event loop'u bloklamadan çalıştırmak; ayıklanan videoyu tarayıcıdan erişilebilir kılmak.

Yaklaşım

1. `scripts/` modüllerinin import edilmesi: `sys.path` bootstrap

`scripts/` bir Python paketi değil; kendi içinde birbirini düz (bare) isimlerle import ediyor (örn. `detect_trailer.py` içinde `from make_polyglot import ...`). Bu yapıyı 1-2. hafta boyunca test edilmiş haliyle bozmadan yeniden kullanmak için `backend/app/pipeline.py` 'nin en başında `scripts/` dizini `sys.path` 'e eklenip modüller olduğu gibi import edildi:

```
SCRIPTS_DIR = Path(__file__).resolve().parent.parent.parent / "scripts"
sys.path.insert(0, str(SCRIPTS_DIR))
import detect_trailer, entropy, extract, size_analysis, video_metadata
```

Plan'ın "gerekirse paketleştirilir/taşınır" notu değerlendirildi; script'leri paketleştirmek (her dosyaya görelî import eklemek) hem 1-2. haftanın CLI kullanım şeklini (`python scripts/detect_trailer.py --file ...`) bozma riski taşıyordu hem de bu günün kapsamı için gereksiz bir refactor olurdu. `sys.path` bootstrap'i her iki kullanım şeklini de (CLI ve API import'u) bozmadan çalıştırıyor.

2. `run_pipeline` : `scripts/analyze.py` 'nin API'ye taşınmış hali

`pipeline.run_pipeline(saved_path)` fonksiyonu Gün 4/6/8/9'da yazılan trailer tespiti, boyut sapma analizi, entropy hesabı adımlarını (`scripts/analyze.py` 'deki mantığın aynısı) birleştirir; ek olarak dosya polyglot ise Gün 8'in `extract.extract()` fonksiyonunu ve Gün 9'un `video_metadata.get_metadata()` fonksiyonunu çağırıp sonucu tek bir sözlükte döner. Dosya

polyglot değilse extraction adımı atlanır (zaten `extract.extract()` polyglot olmayan dosyalarda `ValueError` fırlatıyor).

3. Event loop'u bloklamama: `asyncio.to_thread`

İki seçenek değerlendirildi: - **BackgroundTasks** : yanıt gönderildikten *sonra* çalışır — endpoint'in analiz sonucunu (`polyglot_status`, `extracted_video_url`) response body'de dönmesi gerektiğinden (Gün 13 kabul kriteri: "istek bloklanmadan analiz tamamlanıyor", yani sonuç yine aynı response'ta bekleniyor) bu seçenek uygun değildi. - **`asyncio.to_thread`** (seçilen): `run_pipeline` senkron bir fonksiyon olarak kalır, çağrı `await asyncio.to_thread(pipeline.run_pipeline, saved_path)` ile ayrı bir thread'de yürütülür; event loop bu süre boyunca başka istekleri işlemeye devam edebilir, ama endpoint yine de sonucu bekleyip aynı response'ta döner.

backend/app/main.py :

```
result = await asyncio.to_thread(pipeline.run_pipeline, saved_path)
return AnalyzeResponse(
    polyglot_status=result["polyglot_status"],
    extracted_video_url=result["extracted_video_url"],
    analysis_summary=result["trailer"]["analysis_summary"],
)
```

`threat_score` ve ağırlıklı/dinamik `analysis_summary` Gün 14'e bırakıldı (plan'da bu adım açıkça Gün 14'e ayrılmış); bu yüzden şimdilik `analysis_summary` doğrudan Gün 4'ün trailer özetini taşıyor, `threat_score` `None` .

4. Statik video sunumu: `StaticFiles`

`pipeline.MEDIA_DIR` (`backend/app/media/`) `app.mount("/media", StaticFiles(directory=...), name="media")` ile bağlandı; `run_pipeline` extraction sonrası video dosyasını doğrudan bu dizine yazıyor (`extract.extract(saved_path, MEDIA_DIR)`), yani ekstra bir kopyalama adımına gerek kalmadı. Endpoint yanıtındaki `extracted_video_url` alanı `/media/<uuid>_extracted.mp4` formatında dönüyor.

5. `UploadAck` 'in kaldırılması

Gün 12 raporunda not edildiği gibi `UploadAck` yalnızca Gün 14'e kadar geçici bir modeldi; artık `AnalyzeResponse` (`polyglot_status` + `extracted_video_url` alanlarıyla) tam olarak devrede olduğundan `UploadAck` `models.py` 'den kaldırıldı (kullanılmayan kod bırakılmadı).

Dosyalar

```
backend/app/pipeline.py    # yeni: scripts/ köprüsü + run_pipeline()
backend/app/main.py        # endpoint pipeline'a bağlandı, /media mount edildi
backend/app/models.py      # UploadAck kaldırıldı, AnalyzeResponse docstring güncellendi
```

Test Sonuçları

uvicorn app.main:app --port 8013 ile başlatılıp curl ile test edildi.

Senaryo	Beklenen	Sonuç
<code>samples/sample.png</code> (temiz)	<code>polyglot_status: false ,</code> <code>extracted_video_url:</code> <code>null</code>	✓
<code>samples/polyglot_png.png</code>	<code>polyglot_status: true ,</code> <code>extracted_video_url</code> dolu	✓ <code>/media/<uuid>_extracted.mp4</code>
<code>samples/polyglot_jpg.jpg</code>	<code>polyglot_status: true ,</code> <code>extracted_video_url</code> dolu	✓
GET <code>/media/<uuid>_extracted.mp4</code>	200, <code>video/mp4</code> , <code>ffprobe</code> ile açılabilir	✓ 200 , <code>duration=2.000000</code> (<code>ffprobe</code>)
5 eşzamanlı POST <code>/api/v1/analyze</code> sırasında GET <code>/</code>	event loop bloklanmamalı, hızlı dönmeli	✓ GET / 5 eşzamanlı analiz isteği sürerken <code>0.014s</code> 'de yanıt verdi

```
$ curl -s -F "file=@samples/polyglot_png.png;type=image/png" http://127.0.0.1:8013/api/v1/analyze
{"polyglot_status":true,"threat_score":null,"extracted_video_url":"/media/4e9537fe1a234d3a95d5cd034...

$ curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://127.0.0.1:8013/media/4e9537fe1a23...
200 video/mp4
```

Kabul Kriterleri — Durum

- [x] `/api/v1/analyze` endpoint'i çağrıldığında istek bloklanmadan (event loop donmadan) analiz tamamlanıyor
- [x] Ayıklanan video `/media/...` yolundan tarayıcıda erişilebiliyor

Notlar / Riskler

- `MAX_UPLOAD_SIZE` doğrulaması (Gün 12) ve magic-bytes kontrolü değiştirilmedi; pipeline entegrasyonu bu doğrulamalardan sonra devreye giriyor.
- `backend/app/media/` dosyaları hiçbir zaman silinmiyor (her istek uuid bazlı benzersiz dosya adı üretiyor); üretim ortamı için bir temizleme (TTL/cron) mekanizması gerekir, ancak bu projenin eğitim/demo kapsamının dışında tutuldu.
- `threat_score` ve dinamik `analysis_summary` Gün 14'te işlenecek.